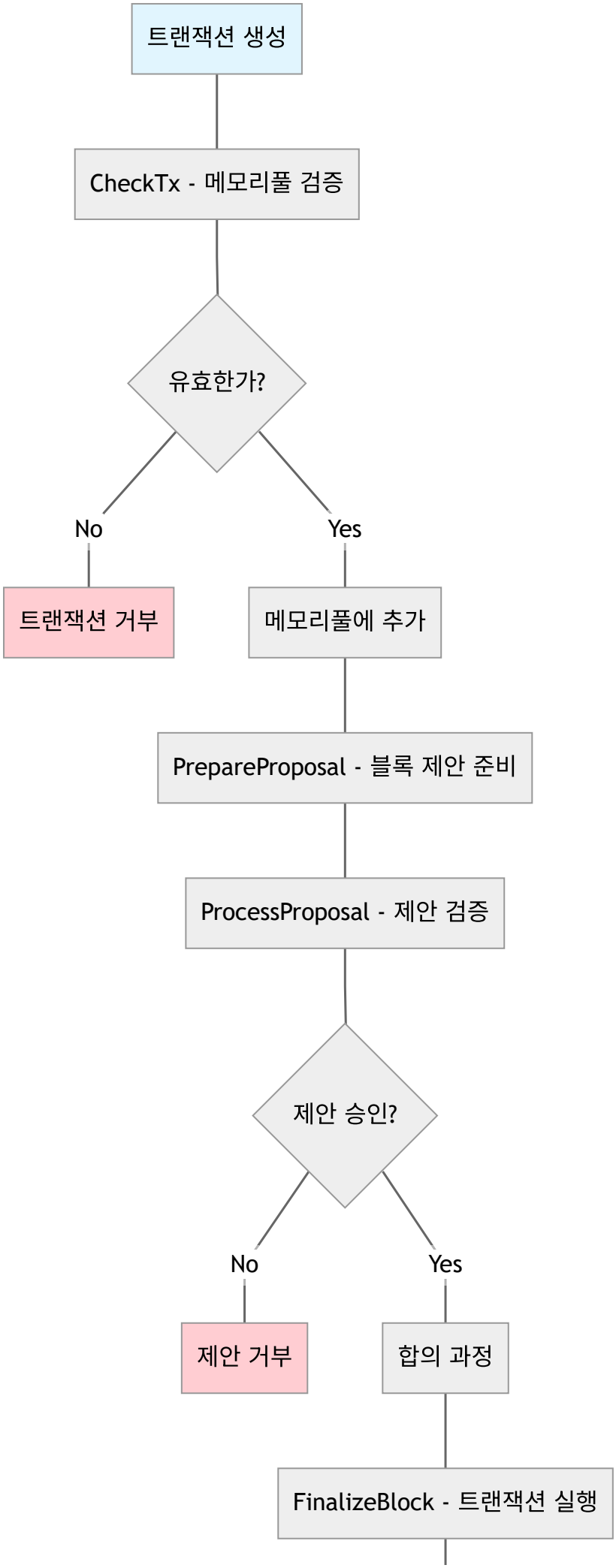
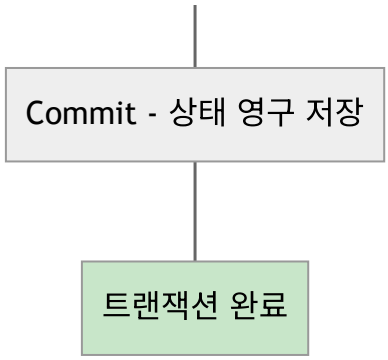
애플리케이션 생명주기

개요

ABCI 애플리케이션의 생명주기는 **트랜잭션이 생성되어 블록체인에 영구적으로 기록되기까지의 전체 과정**을 의미합니다. 이 과정은 여러 단계로 구성되며, 각 단계에서 특정한 ABCI 메서드가 호출되어 트랜잭션의 유효성을 검증하고 상태를 변경합니다.

전체 생명주기 개요


트랜잭션 생성
CheckTx - 메모리풀 검증
유효한가?
No
Yes
트랜잭션 거부
메모리풀에 추가
PrepareProposal - 블록 제안 준비
ProcessProposal - 제안 검증
제안 승인?
No
Yes
제안 거부
합의 과정
FinalizeBlock - 트랜잭션 실행



단계별 상세 분석

1. 트랜잭션 생성 및 전파

과정:

1. 클라이언트가 트랜잭션을 생성
2. 트랜잭션이 P2P 네트워크를 통해 전파
3. 각 노드의 메모리풀이 트랜잭션을 수신

관련 컴포넌트:

- 클라이언트 애플리케이션
- P2P 네트워크
- 메모리풀

2. CheckTx - 메모리풀 검증 단계

목적: 트랜잭션이 메모리풀에 포함될 자격이 있는지 검증

호출 시점:

- 새로운 트랜잭션이 메모리풀에 도착했을 때
- 블록 커밋 후 메모리풀 재검증 시 (Recheck)

검증 내용:

```
func (app *MyApplication) CheckTx(ctx context.Context, req *CheckTxRequest) (*Ch
```

```go
tx, err := parseTx(req.Tx)
if  err != nil {
return &CheckTxResponse{Code: 1, Log: "Parse error"}, nil
}
// 1. 형식 검증
if !isValidFormat(tx) {
return &CheckTxResponse{Code: 2, Log: "Invalid format"}, nil
}
// 2. 서명 검증
if !verifySignature(tx) {
return &CheckTxResponse{Code: 3, Log: "Invalid signature"}, nil
}
// 3. 기본 비즈니스 로직 검증 (상태 변경 없이)
if !hasValidNonce(tx, app.state) {
return &CheckTxResponse{Code: 4, Log: "Invalid nonce"}, nil
}
if !hasSufficientBalance(tx, app.state) {
return &CheckTxResponse{Code: 5, Log: "Insufficient balance"}, nil
}
return &CheckTxResponse{
Code: 0,
GasWanted: tx.GasLimit,
GasUsed: estimateGas(tx),
```

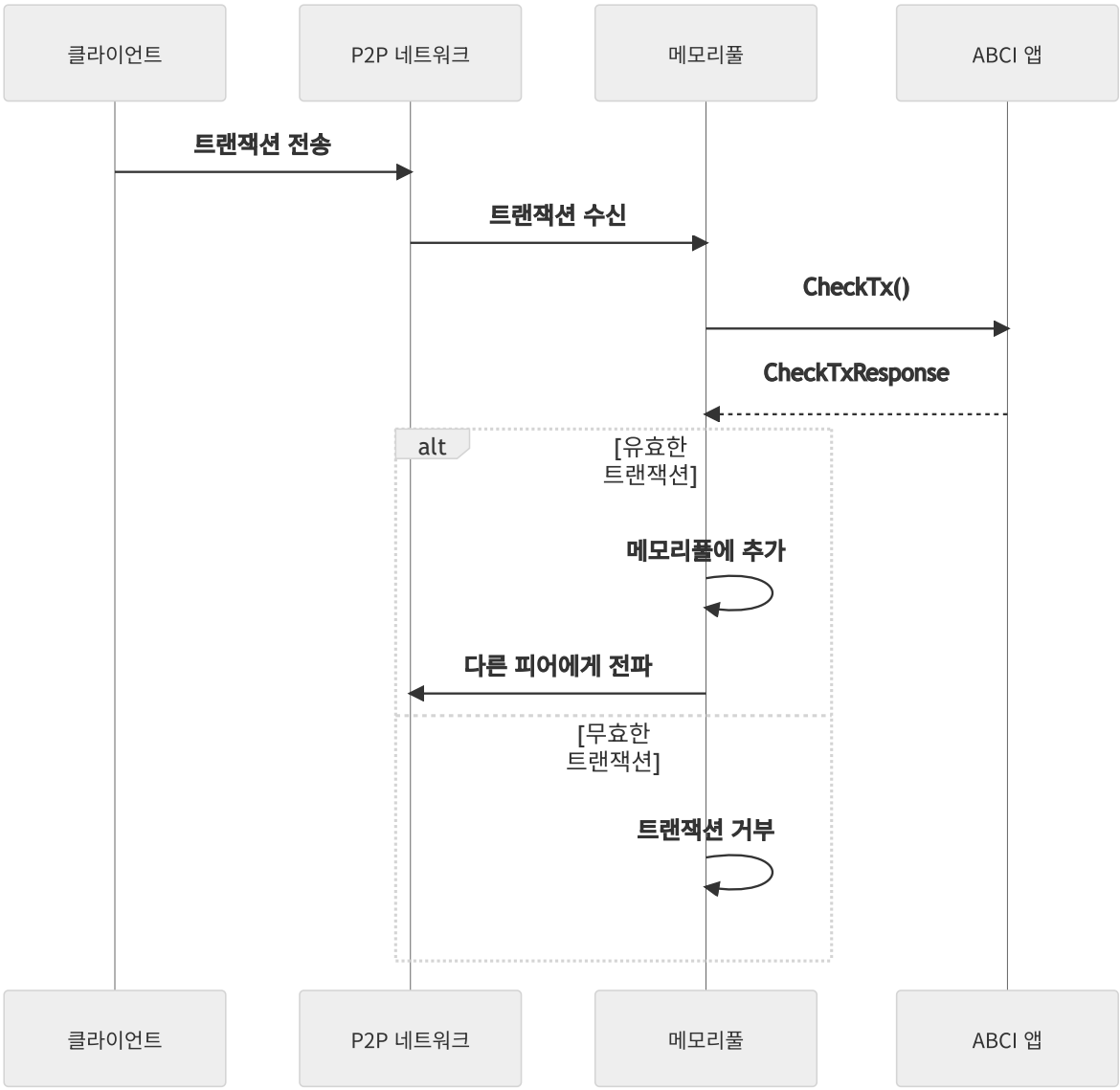
```
}, nil

}
```

중요 원칙:

- **상태를 변경하지 않음**: 읽기 전용 검증만 수행
- **빠른 실행**: 메모리풀 성능에 직접적 영향
- **결정론적 결과**: 모든 노드에서 동일한 결과 보장

3. 메모리풀 관리

과정:



메모리풀 특성:

- **우선순위 기반 정렬**: 가스 가격, 수수료 등에 따른 우선순위
- **크기 제한**: 메모리 사용량 제한
- **재검증**: 블록 커밋 후 남은 트랜잭션들의 유효성 재확인

4. PrepareProposal - 블록 제안 준비

목적: 새로운 블록에 포함할 트랜잭션들을 선택하고 정렬

호출 시점: 밸리데이터가 새 블록을 제안할 차례일 때

구현 예제:

```
func (app *MyApplication) PrepareProposal(ctx context.Context, req *PrepareProposal

var selectedTxs [][]byte

var totalBytes int64

// 1. 트랜잭션 우선순위 정렬

sortedTxs := app.sortTxsByPriority(req.Txs)

// 2. 크기 제한 내에서 트랜잭션 선택

for _, tx := range sortedTxs {

txSize := int64(len(tx))

if totalBytes+txSize > req.MaxTxBytes {

break

}

// 3. 재검증 (선택사항)

if app.shouldRevalidate(tx) {

if !app.isStillValid(tx) {

continue

}

}

selectedTxs = append(selectedTxs, tx)

totalBytes += txSize
```

```
}

// 4. 애플리케이션별 특수 트랜잭션 추가 (예: 시스템 트랜잭션)

systemTxs := app.generateSystemTxs(req.Height, req.Time)

selectedTxs = append(selectedTxs, systemTxs...)

return &PrepareProposalResponse{

Txs: selectedTxs,

}, nil

}
```

고려사항:

- **가스 한도**: 블록 가스 한도 내에서 트랜잭션 선택
- **우선순위**: 수수료, 중요도에 따른 트랜잭션 정렬
- **특수 트랜잭션**: 시스템 레벨 트랜잭션 포함 가능

5. ProcessProposal - 제안 검증

목적: 다른 밸리데이터의 블록 제안을 검증

호출 시점: 다른 밸리데이터로부터 블록 제안을 받았을 때

검증 로직:

```
func (app *MyApplication) ProcessProposal(ctx context.Context, req *ProcessPropo

// 1. 블록 크기 검증

totalSize := int64(0)

for  _, tx := range  req.Txs {

totalSize += int64(len(tx))
```

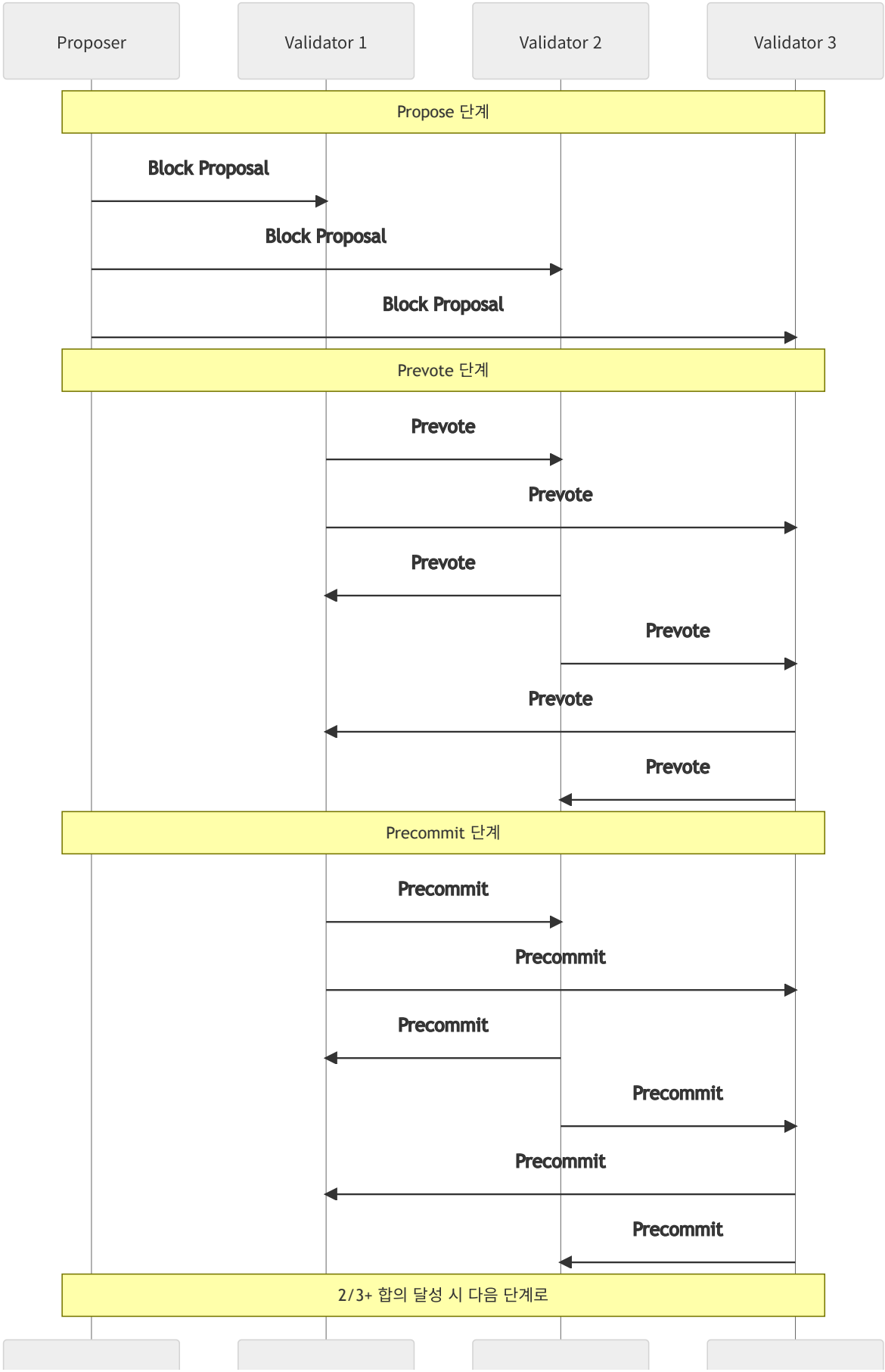
```
}
if  totalSize > app.maxBlockSize {
return &ProcessProposalResponse{
Status: PROCESS_PROPOSAL_STATUS_REJECT,
}, nil
}
// 2. 각 트랜잭션 유효성 검증
for  _, tx := range  req.Txs {
if !app.isValidTxForBlock(tx, req.Height) {
return &ProcessProposalResponse{
Status: PROCESS_PROPOSAL_STATUS_REJECT,
}, nil
}
}
// 3. 블록 레벨 검증
if !app.isValidBlockStructure(req) {
return &ProcessProposalResponse{
Status: PROCESS_PROPOSAL_STATUS_REJECT,
}, nil
}
return &ProcessProposalResponse{
Status: PROCESS_PROPOSAL_STATUS_ACCEPT,
}, nil
```

```
}
```

6. 합의 과정

Tendermint 합의 단계:

1. **Propose**: 블록 제안
2. **Prevote**: 제안에 대한 사전 투표
3. **Precommit**: 최종 커밋 투표
4. **Commit**: 블록 확정


Proposer
Validator 1
Validator 2
Validator 3
Propose 단계
Block Proposal
Block Proposal
Block Proposal
Prevote 단계
Prevote
Prevote
Prevote
Prevote
Prevote
Prevote
Precommit 단계
Precommit
Precommit
Precommit
Precommit
Precommit
Precommit
2/3+ 합의 달성 시 다음 단계로

Proposer | Validator 1 | Validator 2 | Validator 3

7. FinalizeBlock - 트랜잭션 실행

목적: 합의된 블록의 트랜잭션들을 실제로 실행하고 상태를 변경

호출 시점: 블록이 합의에 도달한 후

실행 과정:

```
func (app *MyApplication) FinalizeBlock(ctx context.Context, req *FinalizeBlockR

var  txResults []ExecTxResult

var  blockEvents []Event

// 1. 블록 시작 처리

app.beginBlock(req.Height, req.Time)

// 2. 각 트랜잭션 순차 실행

for  i, txBytes := range  req.Txs {

// 트랜잭션 실행

result := app.executeTx(txBytes, req.Height)

txResults = append(txResults, result)

// 실행 실패 시에도 결과 기록 (상태는 롤백)

if  result.Code != 0 {

app.logger.Info("Transaction failed", "index", i, "code", result.Code, "log", re

}
```

```go
// 이벤트 수집
blockEvents = append(blockEvents, result.Events...)
}
// 3. 블록 종료 처리
app.endBlock(req.Height)
// 4. 상태 해시 계산
appHash := app.computeAppHash()
// 5. 밸리데이터 업데이트 (필요시)
validatorUpdates := app.getValidatorUpdates()
return &FinalizeBlockResponse{
Events: blockEvents,
TxResults: txResults,
ValidatorUpdates: validatorUpdates,
AppHash: appHash,
}, nil
}
```

트랜잭션 실행 세부사항:

```go
func (app *MyApplication) executeTx(txBytes []byte, height  int64) ExecTxResult
// 1. 트랜잭션 파싱
tx, err := parseTx(txBytes)
if  err != nil {
return ExecTxResult{Code: 1, Log: "Parse error"}
```

```
}
// 2. 가스 추적 시작
gasTracker := NewGasTracker(tx.GasLimit)
// 3. 기본 검증 (가스 소모)
if !gasTracker.ConsumeGas(BasicValidationGas) {
return ExecTxResult{Code: 2, Log: "Out of gas"}
}
// 4. 서명 검증 (가스 소모)
if !app.verifySignature(tx, gasTracker) {
return ExecTxResult{Code: 3, Log: "Invalid signature"}
}
// 5. 상태 변경 실행
events, err := app.applyTx(tx, gasTracker)
if  err != nil {
return ExecTxResult{
Code: 4,
Log: err.Error(),
GasWanted: tx.GasLimit,
GasUsed: gasTracker.GasUsed(),
}
}
return ExecTxResult{
Code: 0,
```

```
Events: events,

GasWanted: tx.GasLimit,

GasUsed: gasTracker.GasUsed(),

}

}
```

8. Commit - 상태 영구 저장

목적: 변경된 상태를 디스크에 영구적으로 저장

호출 시점: FinalizeBlock 실행 완료 후

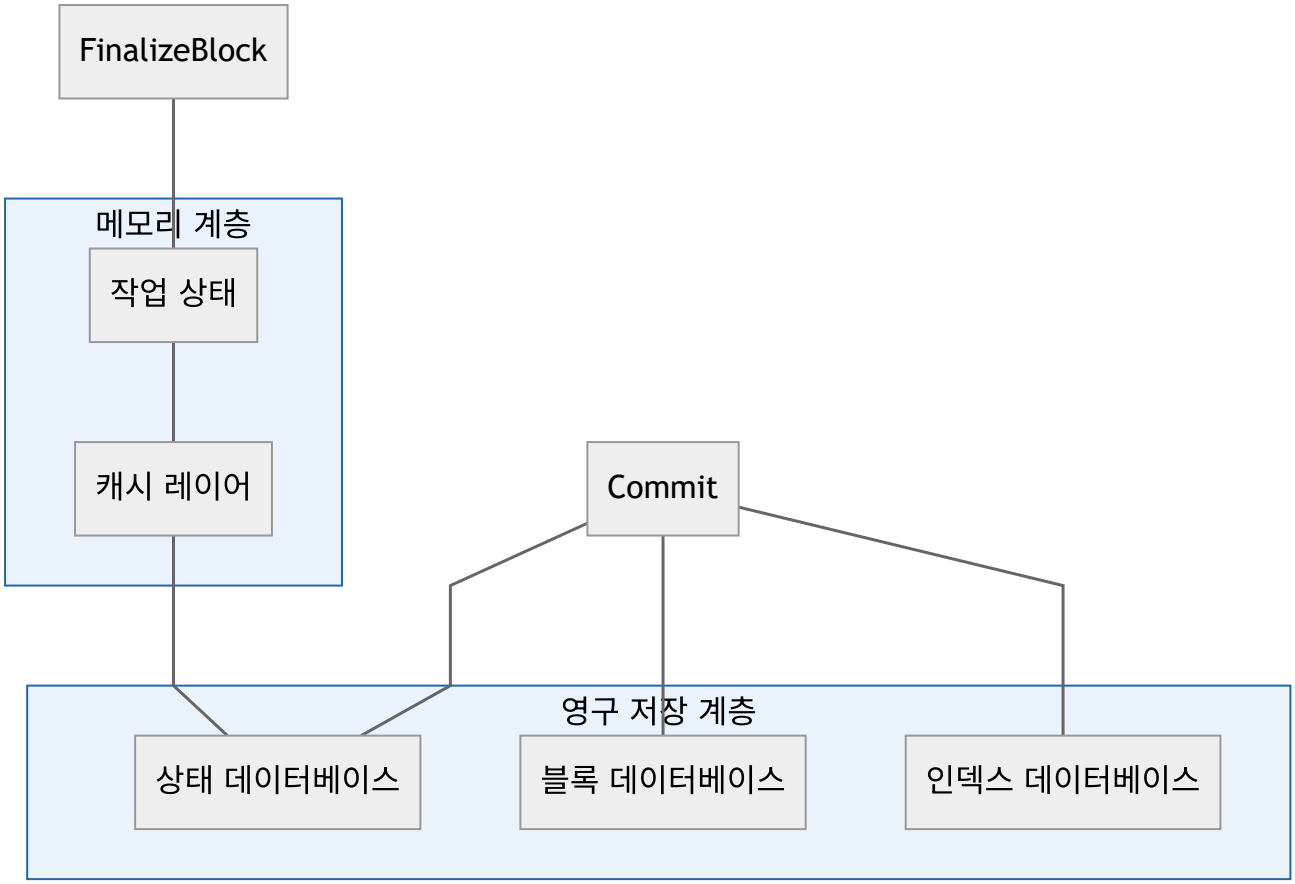
구현 예제:

```
func (app *MyApplication) Commit(ctx context.Context, req *CommitRequest) (*Comm

// 1. 메모리 상태를 디스크에 저장

err := app.state.Commit()

if  err != nil {

return  nil, fmt.Errorf("failed to commit state: %w", err)

}

// 2. 인덱스 업데이트

err = app.indexer.UpdateIndexes(app.state.Height)

if  err != nil {

app.logger.Error("Failed to update indexes", "error", err)

}

// 3. 메트릭 업데이트
```

```
app.updateMetrics()

// 4. 오래된 데이터 정리 정책

retainHeight := app.calculateRetainHeight()

// 5. 메모리풀 정리 (커밋된 트랜잭션 제거)

app.cleanupMempool()

return &CommitResponse{

RetainHeight: retainHeight,

}, nil

}
```

상태 변경과 커밋 과정

상태 관리 아키텍처



상태 변경 흐름

1. **FinalizeBlock 시작**:

- 새로운 작업 상태 생성
- 이전 블록 상태를 기반으로 초기화

2. **트랜잭션 실행**:

- 각 트랜잭션이 작업 상태를 수정
- 실패한 트랜잭션은 상태 변경 롤백

3. **블록 완료**:

- 모든 트랜잭션 실행 완료
- 최종 상태 해시 계산

4. **Commit 실행**:

- 작업 상태를 영구 저장소에 기록

- 트랜잭션 원자성 보장

트랜잭션 원자성

```
func (app *MyApplication) applyTx(tx Transaction, gasTracker *GasTracker) ([]Eve

// 상태 스냅샷 생성

snapshot := app.state.CreateSnapshot()

defer func() {

if r := recover(); r != nil {

// 패닉 발생 시 상태 롤백

app.state.RestoreSnapshot(snapshot)

}

}()

// 트랜잭션 로직 실행

events, err := app.executeTransactionLogic(tx, gasTracker)

if err != nil {

// 에러 발생 시 상태 롤백

app.state.RestoreSnapshot(snapshot)

return nil, err

}

// 성공 시 스냅샷 해제

snapshot.Release()

return events, nil
```

```
}
```

에러 처리 및 복구

트랜잭션 레벨 에러

```
// 트랜잭션 실행 중 발생할 수 있는 에러들
const (
CodeOK = 0
CodeInvalidFormat = 1
CodeInvalidSignature = 2
CodeInsufficientFunds = 3
CodeInvalidNonce = 4
CodeOutOfGas = 5
CodeUnknownRequest = 6
CodeInvalidAddress = 7
CodeUnauthorized = 8
CodeInternalError = 99
)
```

블록 레벨 복구

```
func (app *MyApplication) handleBlockError(height  int64, err  error) {
```

```
app.logger.Error("Block execution failed",
"height", height,
"error", err)
// 상태를 이전 블록으로 롤백
app.state.RollbackToHeight(height - 1)
// 메트릭 업데이트
app.metrics.BlockErrors.Inc()
// 알림 발송 (선택사항)
app.notifyError(height, err)
}
```

성능 최적화 고려사항

1. 병렬 처리

- **CheckTx 병렬화**: 독립적인 트랜잭션 검증
- **상태 읽기 최적화**: 캐시 활용

2. 메모리 관리

- **상태 캐싱**: 자주 접근하는 데이터 메모리 보관
- **가비지 컬렉션**: 오래된 상태 정리

3. 디스크 I/O 최적화

- **배치 쓰기**: 여러 변경사항을 한 번에 저장
- **압축**: 상태 데이터 압축 저장

모니터링 및 디버깅

주요 메트릭

```
type ApplicationMetrics struct {
// 트랜잭션 관련
TxProcessed prometheus.Counter
TxFailed prometheus.Counter
TxGasUsed prometheus.Histogram
// 블록 관련
BlockHeight prometheus.Gauge
BlockSize prometheus.Histogram
BlockTime prometheus.Histogram
// 상태 관련
StateSize prometheus.Gauge
CommitTime prometheus.Histogram
}
```

로깅 전략

```
func (app *MyApplication) logTransactionExecution(tx Transaction, result ExecTxR
fields := map[string]interface{}{
"tx_hash": tx.Hash(),
"tx_type": tx.Type(),
```

```
"result_code": result.Code,

"gas_used": result.GasUsed,

"gas_wanted": result.GasWanted,

}

if  result.Code == 0 {

app.logger.Info("Transaction executed successfully", fields)

} else {

fields["error"] = result.Log

app.logger.Warn("Transaction execution failed", fields)

}

}
```

다음 단계

애플리케이션 생명주기를 이해했다면, 다음 문서를 통해 실제 구현 방법을 학습할 수 있습니다:

- ABCI 실습 가이드: 실제 애플리케이션 구현 예제
- 상태 관리: 상태 저장소 구조와 관리 방법
- 성능 최적화: 애플리케이션 성능 최적화 방법

이 생명주기의 각 단계를 정확히 이해하고 구현하는 것이 안정적이고 효율적인 블록체인 애플리케이션 개발의 핵심입니다.